

CSE225 Multiple Choice Question Bank

Stack & Queue (Array), Stack & Queue (Linked List), Recursion

Source: Lecture 08, Lecture 09 and Lecture 10 slides only.

Total questions: 221, arranged in 8 sections.

Coverage: definitions, specifications, code details, keyword-level questions, time complexity, traces and applications.

Answer key with explanations begins after the last section.

How to use this: work through a section with the answer key covered, then check. Every explanation names the exact slide idea it comes from, so a wrong answer tells you which sub-topic to re-read.

SECTION A - Stack: ADT Specification & Array-Based Implementation

1. A stack is best described as a list that maintains which order?

- (A) First In First Out (FIFO)
- (B) Last In First Out (LIFO)
- (C) Highest Priority First Out
- (D) Random access order

2. In the structure of StackType, elements are added to and removed from which position?

- (A) The bottom of the stack
- (B) The middle of the stack
- (C) The top of the stack
- (D) Either end of the stack

3. In the specification of StackType, MAX_ITEMS and ItemType are:

- (A) Operations provided by the ADT
- (B) Definitions provided by the user
- (C) Private data members only
- (D) Exception classes

4. What does MAX_ITEMS represent?

- (A) The current number of items on the stack
- (B) The maximum number of items that might be on the stack
- (C) The size in bytes of one item
- (D) The index of the top item

5. What does ItemType represent in the StackType specification?

- (A) The data type of the items on the stack
- (B) The type of the top index
- (C) The exception type thrown on overflow
- (D) The return type of IsEmpty

6. Which operation of StackType has a postcondition but no precondition listed?

- (A) IsEmpty
- (B) Push
- (C) MakeEmpty
- (D) Top

7. What is the precondition shared by IsEmpty, IsFull, Push, Pop and Top?

- (A) The stack must be non-empty
- (B) The stack has been initialized
- (C) The stack must be non-full
- (D) ItemType must be a primitive type

8. According to the specification, the postcondition of Push when the stack is full is:

- (A) newItem overwrites the top element
- (B) The stack is silently unchanged
- (C) Exception FullStack is thrown
- (D) MakeEmpty is called automatically

9. Which two operations can throw the EmptyStack exception?

- (A) Push and MakeEmpty
- (B) Pop and Top
- (C) IsEmpty and IsFull
- (D) Push and Pop

10. What is the return type of the Top() operation?

- (A) void
- (B) bool
- (C) ItemType
- (D) ItemType&

11. What is the essential difference between Pop() and Top()?

- (A) Pop returns the item, Top removes it
- (B) Pop removes the top item, Top returns a copy of it without removing
- (C) They are identical operations
- (D) Pop works on queues, Top works on stacks

12. In stacktype.h, what is the declared value of the constant MAX_ITEMS?

- (A) 1
- (B) 5
- (C) 100
- (D) 500

13. How are FullStack and EmptyStack defined in stacktype.h?

- (A) As enum values
- (B) As empty classes used as exception classes
- (C) As integer error codes
- (D) As member functions of StackType

14. What are the private data members of the array-based StackType?

- (A) ItemType* items and int size
- (B) int top and ItemType items[MAX_ITEMS]
- (C) NodeType* topPtr
- (D) int front and int rear

15. What is the purpose of the #ifndef / #define / #endif pair in stacktype.h?

- (A) To make the class a template
- (B) To act as an include guard preventing multiple inclusion
- (C) To enable exception handling
- (D) To reserve memory for the array

16. The line 'template <class ItemType>' placed above class StackType means:

- (A) The class can only store integers
- (B) The class is generic and works with any item type supplied at compile time
- (C) The class is abstract and cannot be instantiated
- (D) ItemType is a global variable

17. Why does the array-based StackType in Lecture 08 not declare a destructor?

- (A) Destructors are illegal in template classes
- (B) The items array is a static member array, so no dynamic memory is allocated
- (C) The compiler forbids it when exceptions are used
- (D) Because top is initialised to -1

18. What value does the StackType constructor assign to top?

- (A) 0
- (B) -1
- (C) MAX_ITEMS
- (D) MAX_ITEMS - 1

19. Why is top initialised to -1 rather than 0?

- (A) Because arrays in C++ start at 1
- (B) Because -1 signals that no element occupies index 0 yet; the first Push makes top 0
- (C) Because -1 is the value of NULL
- (D) To reserve items[0] as a dead slot

20. What is the body of IsEmpty() in the array implementation?

- (A) return (top == 0);
- (B) return (top == -1);
- (C) return (items == NULL);
- (D) return (top == MAX_ITEMS);

21. What is the body of IsFull() in the array implementation?

- (A) return (top == MAX_ITEMS);
- (B) return (top == MAX_ITEMS - 1);
- (C) return (top + 1 == 0);
- (D) return (top % MAX_ITEMS == 0);

22. What is the body of MakeEmpty() in the array implementation?

- (A) delete [] items;
- (B) top = -1;
- (C) for each element: items[i] = 0;
- (D) top = MAX_ITEMS - 1;

23. In Push, what is the correct order of the two final statements?

- (A) items[top] = newItem; then top++;
- (B) top++; then items[top] = newItem;
- (C) top--; then items[top] = newItem;
- (D) The order does not matter

24. What is the first thing Push does before storing the new item?

- (A) Calls MakeEmpty()
- (B) Checks if(IsFull()) and throws FullStack()
- (C) Checks if(IsEmpty()) and throws EmptyStack()
- (D) Increments MAX_ITEMS

25. What does Pop() actually do to the popped element in the array implementation?

- (A) Overwrites it with zero
- (B) Calls delete on it
- (C) Only decrements top, leaving the old value physically in the array but unreachable
- (D) Shifts every element left by one

26. What does Top() return?

- (A) items[0]
- (B) items[top]
- (C) items[top-1]
- (D) top

27. A stack with MAX_ITEMS = 5 is empty. After Push('A'), Push('B'), Push('C'), what is the value of top?

- (A) 0
- (B) 2
- (C) 3
- (D) -1

28. For MAX_ITEMS = 5, the stack is full when top equals:

- (A) 3
- (B) 4
- (C) 5
- (D) 6

29. Calling Pop() on a stack whose top is -1 results in:

- (A) top becoming -2
- (B) A FullStack exception
- (C) An EmptyStack exception
- (D) Undefined silent behaviour

30. What is the time complexity of Push in the array-based stack?

- (A) $O(1)$
- (B) $O(\log N)$
- (C) $O(N)$
- (D) $O(N \log N)$

31. Which of the array-based stack operations is NOT $O(1)$?

- (A) MakeEmpty
- (B) Pop
- (C) Top
- (D) None - all of them are $O(1)$

32. Why is MakeEmpty $O(1)$ rather than $O(N)$ in the array-based stack?

- (A) Because the array is small
- (B) Because it only assigns $\text{top} = -1$ and does not touch the stored elements
- (C) Because the compiler optimises the loop away
- (D) Because it calls Pop repeatedly

SECTION B - Applications of Stack

33. What is step 1 of the algorithm for matching a parentheses string?

- (A) Read the whole string into an array
- (B) Initialise an empty stack
- (C) Count the opening parentheses
- (D) Reverse the string

34. In the parentheses-matching algorithm, what is done when an opening parenthesis is read?

- (A) It is popped from the stack
- (B) It is pushed into the stack
- (C) It is ignored
- (D) It is compared with the top of the stack

35. In the parentheses-matching algorithm, what is done when a closing parenthesis is read?

- (A) It is pushed into the stack
- (B) A pop is performed on the stack
- (C) The stack is emptied
- (D) The algorithm halts immediately

36. After the whole string has been processed, an EMPTY stack indicates:

- (A) An unbalanced string
- (B) A balanced string of parentheses
- (C) An invalid character in the string
- (D) That the string was empty

37. After the whole string has been processed, a NON-EMPTY stack indicates:

- (A) A balanced string
- (B) An unbalanced string of parentheses
- (C) That more items must be read
- (D) A stack overflow

38. During processing, an unsuccessful pop (popping from an empty stack) indicates:

- (A) A balanced string
- (B) An unbalanced string of parentheses
- (C) The string has ended
- (D) The stack size must be increased

39. Which of these strings is balanced?

- (A) $((()))((())$
- (B) $((()))()((()$
- (C) $((()))()((())())$
- (D) $((())((()$

40. Which structural property of a stack makes it the natural choice for parenthesis matching?

- (A) It gives random access to any element
- (B) The most recently opened parenthesis is the first one that must be closed (LIFO)
- (C) It automatically sorts its contents
- (D) It has $O(N)$ insertion

41. For the string $((()))()((()$, processing ends with the stack containing:

- (A) Nothing - it is empty
- (B) Two unmatched opening parentheses
- (C) Four unmatched opening parentheses
- (D) One closing parenthesis

42. The main advantage of postfix notation over infix notation is:

- (A) It is shorter to write
- (B) No parentheses are necessary
- (C) It supports more operators
- (D) It can be evaluated without a stack

43. The postfix form of the infix expression $2 - 3 * 4 + 5$ is:

- (A) $2\ 3\ 4\ *\ -\ 5\ +$
- (B) $2\ 3\ -\ 4\ 5\ +\ *$
- (C) $2\ 3\ 4\ *\ 5\ +\ -$
- (D) $2\ 3\ 4\ 5\ *\ +\ -$

44. The infix expression $2 - 3 * 4 + 5$ evaluates to:

- (A) 5
- (B) -5
- (C) 15
- (D) -15

45. The postfix form of the infix expression $(2 - 3) * (4 + 5)$ is:

- (A) $2\ 3\ 4\ *\ -\ 5\ +$
- (B) $2\ 3\ -\ 4\ 5\ +\ *$
- (C) $2\ 3\ 4\ *\ 5\ +\ -$
- (D) $2\ 3\ 4\ +\ 5\ -\ *$

46. The expression $(2 - 3) * (4 + 5)$ evaluates to:

- (A) -5
- (B) -9
- (C) 9
- (D) -15

47. The postfix expression $2\ 3\ 4\ *\ 5\ +\ -$ evaluates to:

- (A) -5
- (B) -9
- (C) -15
- (D) 15

48. According to the slides, which data structure is used to convert an infix expression into postfix form?

- (A) A circular queue
- (B) A binary tree
- (C) A hash table
- (D) A doubly linked list

49. The postfix form of $(4+8)*(6-5)/((3-2)*(2+2))$ is:

- (A) $4\ 8\ +\ 6\ 5\ -\ *\ 3\ 2\ -\ 2\ 2\ +\ *\ /$
- (B) $4\ 8\ +\ 6\ 5\ -\ 3\ 2\ -\ 2\ 2\ +\ **\ /$
- (C) $4\ 8\ +\ *\ 6\ 5\ -\ 3\ 2\ -\ /\ 2\ 2\ +$
- (D) $8\ 4\ +\ 5\ 6\ -\ *\ 2\ 3\ -\ 2\ 2\ +\ *\ /$

50. Evaluating $4\ 8\ +\ 6\ 5\ -\ *\ 3\ 2\ -\ 2\ 2\ +\ *\ /$ gives the final result:

- (A) 1
- (B) 3
- (C) 12
- (D) 48

51. In postfix evaluation, what is done when an operand is read?

- (A) It is pushed into the stack
- (B) It is popped from the stack
- (C) It is added to the previous operand
- (D) It is discarded

52. In postfix evaluation, what is done when an operator is read?

- (A) It is pushed into the stack
- (B) The top item is popped and negated
- (C) The top two items are popped, the operator is applied, and the answer is pushed back
- (D) The stack is emptied

53. At the end of postfix evaluation, exactly one item left in the stack indicates:

- (A) An invalid expression
- (B) A valid expression, and that item is the answer
- (C) That an operator is missing
- (D) That the stack overflowed

54. At the end of postfix evaluation, more than one item in the stack or an unsuccessful pop indicates:

- (A) A valid expression
- (B) An invalid expression
- (C) A balanced expression
- (D) An empty expression

55. While evaluating $4\ 8\ +\ 6\ 5\ -\ *\ \dots$, immediately after the '+' is processed the stack contains:

- (A) 4 and 8
- (B) 12
- (C) 12 and 6
- (D) Nothing

56. While evaluating the same expression, after 12 6 5 and then '-' is processed, the stack contains:

- (A) 12 and 1
- (B) 12, 6 and 5
- (C) 12 and 11
- (D) 1

57. In postfix evaluation, why must the order of the two popped operands be respected?

- (A) It never matters
- (B) Because - and / are not commutative, so swapping the operands changes the result
- (C) Because the stack sorts them
- (D) Because + and * are not commutative

SECTION C - Queue: ADT Specification & Array-Based (Circular) Implementation

58. A queue is a list that maintains which order?

- (A) Last In First Out (LIFO)
- (B) First In First Out (FIFO)
- (C) Sorted order
- (D) Reverse sorted order

59. In the structure of QueueType, elements are:

- (A) Added to the front and removed from the rear
- (B) Added to the rear and removed from the front
- (C) Added and removed at the rear only
- (D) Added and removed at the front only

60. Which operation is present in StackType but absent from the QueueType specification?

- (A) MakeEmpty
- (B) IsFull
- (C) Top
- (D) IsEmpty

61. What is the signature of the Dequeue operation?

- (A) ItemType Dequeue()
- (B) void Dequeue(ItemType item)
- (C) void Dequeue(ItemType& item)
- (D) bool Dequeue(ItemType* item)

62. Why is the parameter of Dequeue passed by reference?

- (A) To save memory
- (B) Because the function returns void and must pass the removed element back through the parameter
- (C) Because ItemType is always a class
- (D) To allow the queue to be resized

63. What is the postcondition of Dequeue when the queue is empty?

- (A) item is set to zero
- (B) The EmptyQueue exception is thrown and item is undefined
- (C) The FullQueue exception is thrown
- (D) The queue is silently reinitialised

64. What is the postcondition of Enqueue when the queue is full?

- (A) The oldest item is overwritten
- (B) The FullQueue exception is thrown
- (C) The array grows automatically
- (D) The EmptyQueue exception is thrown

65. If a queue is implemented by always inserting at the back of the array and deleting from index 0 by shifting, what is the complexity of deletion?

- (A) $O(1)$
- (B) $O(\log N)$
- (C) $O(N)$
- (D) $O(N^2)$

66. What is the drawback of the simple 'two indices, front++ and rear++' scheme?

- (A) It cannot store characters
- (B) Dead space accumulates at the front of the array, which can never be reused
- (C) Enqueue becomes $O(N)$
- (D) front and rear must be equal at all times

67. Which formula is used for Enqueue so that the indices wrap around?

- (A) $\text{rear} = \text{rear} + 1;$
- (B) $\text{rear} = (\text{rear} + 1) \% \text{maxQue};$
- (C) $\text{rear} = (\text{rear} - 1) \% \text{maxQue};$
- (D) $\text{rear} = \text{maxQue} \% (\text{rear} + 1);$

68. Which formula is used for Dequeue so that the indices wrap around?

- (A) $\text{front} = \text{front} - 1;$
- (B) $\text{front} = (\text{front} + 1) \% \text{maxQue};$
- (C) $\text{front} = (\text{front} - 1) \% \text{maxQue};$
- (D) $\text{front} = \text{rear} \% \text{maxQue};$

69. Treating the array as a circular structure solves which problem?

- (A) Distinguishing empty from full
- (B) Reusing the dead space at the beginning of the array
- (C) Reducing the memory used by the queue
- (D) Making Dequeue $O(N)$

70. When both front and rear are simply incremented, why can the empty state not be distinguished from the full state?

- (A) Because rear is always larger than front
- (B) Because both situations can produce the same relative values of front and rear
- (C) Because maxQue is unknown
- (D) Because front is always 0

71. What is the solution adopted in the slides to distinguish the empty state from the full state?

- (A) Keep a separate counter of items
- (B) Let front indicate the index of the array slot PRECEDING the front element and leave that slot reserved
- (C) Set rear to -1 when the queue is empty
- (D) Use two separate arrays

72. What is the cost of the reserved-slot solution?

- (A) Enqueue becomes $O(N)$
- (B) One array slot is always wasted, so the array must be one larger than the capacity
- (C) Dequeue can no longer return the item
- (D) The queue can no longer wrap around

73. The queue is FULL when:

- (A) `front == rear`
- (B) `(rear + 1) % maxQue == front`
- (C) `rear == maxQue - 1`
- (D) `(front + 1) % maxQue == rear`

74. The queue is EMPTY when:

- (A) `front == rear`
- (B) `(rear + 1) % maxQue == front`
- (C) `rear == -1`
- (D) `front == 0` and `rear == 0`

75. In `queuetype.h`, what is the value of `DEFAULT_SIZE`?

- (A) 5
- (B) 100
- (C) 500
- (D) 1000

76. In `queuetype.h`, the line `'typedef char ItemType;'` means:

- (A) `ItemType` is a class
- (B) `ItemType` is an alias for `char` in this version of the code
- (C) The queue can only hold pointers
- (D) `char` is redefined as a template

77. What are the private data members of the array-based `QueType`?

- (A) `int top` and `ItemType items[MAX_ITEMS]`
- (B) `int front`, `int rear`, `ItemType* items`, `int maxQue`
- (C) `NodeType* front` and `NodeType* rear`
- (D) `int front`, `int rear` and `int count`

78. How many constructors does the array-based `QueType` declare?

- (A) One
- (B) Two - a default one and one taking `int max`
- (C) Three
- (D) None

79. In the parameterised constructor, why is `maxQue` set to `max + 1` rather than `max`?

- (A) To leave room for the null terminator
- (B) To account for the one reserved slot that is never filled
- (C) Because arrays are 1-indexed
- (D) To make the queue size a power of two

80. What do both constructors of `QueType` assign to `front` and `rear` initially?

- (A) 0
- (B) -1
- (C) `maxQue - 1`
- (D) `maxQue`

81. What does the QueType destructor do?

- (A) Sets front = rear = -1
- (B) Calls MakeEmpty()
- (C) Executes delete [] items;
- (D) Nothing

82. Why does the array-based QueType need a destructor when the array-based StackType does not?

- (A) Because a queue has two indices
- (B) Because QueType allocates its array dynamically with new ItemType[maxQue]
- (C) Because a queue can be full
- (D) Because ItemType is a typedef

83. What is the body of MakeEmpty() in the array-based QueType?

- (A) delete [] items;
- (B) front = maxQue - 1; rear = maxQue - 1;
- (C) front = 0; rear = 0;
- (D) for(int i=0;i<maxQue;i++) items[i]=0;

84. What is the body of IsEmpty() in the array-based QueType?

- (A) return (front == 0);
- (B) return (rear == front);
- (C) return (items == NULL);
- (D) return ((rear+1)%maxQue == front);

85. Inside Enqueue, what is the correct order of operations?

- (A) items[rear] = newItem; then rear = (rear+1)%maxQue;
- (B) rear = (rear + 1) % maxQue; then items[rear] = newItem;
- (C) front = (front+1)%maxQue; then items[front] = newItem;
- (D) Only items[rear] = newItem;

86. Inside Dequeue, what is the correct order of operations?

- (A) item = items[front]; then front = (front+1)%maxQue;
- (B) front = (front + 1) % maxQue; then item = items[front];
- (C) rear = (rear-1)%maxQue; then item = items[rear];
- (D) item = items[rear]; then rear--;

87. What is the time complexity of Enqueue and Dequeue in the circular array queue?

- (A) Both O(N)
- (B) Enqueue O(1), Dequeue O(N)
- (C) Both O(1)
- (D) Enqueue O(N), Dequeue O(1)

88. What is the time complexity of the array-based QueType destructor?

- (A) O(1)
- (B) O(N)
- (C) O(log N)
- (D) O(N^2)

89. A queue is created with QueType q(4). What is the value of maxQue, and how many items can actually be stored?

- (A) maxQue = 4, capacity 4
- (B) maxQue = 5, capacity 4
- (C) maxQue = 5, capacity 5
- (D) maxQue = 4, capacity 3

90. With maxQue = 5, front = 2 and rear = 1, what is the state of the queue?

- (A) Empty
- (B) Full
- (C) Contains exactly one item
- (D) Invalid state

91. With maxQue = 5, front = 3 and rear = 3, what is the state of the queue?

- (A) Full
- (B) Empty
- (C) Contains three items
- (D) Contains one item

SECTION D - Stack: Linked-List-Based Implementation

92. What is the main limitation of the array-based stack, whether the array is static or dynamic?

- (A) It can only hold characters
- (B) You must anticipate the size of the stack before you use it
- (C) Push becomes $O(N)$
- (D) It cannot throw exceptions

93. In the statement 'char *itemPtr = new char;', where is the allocated memory located?

- (A) On the run-time stack
- (B) In the free store (heap)
- (C) In the code segment
- (D) In a CPU register

94. Why is an array of pointers rejected as the solution for a growable stack?

- (A) Pointers cannot be stored in arrays
- (B) You would still need to know the size of that array beforehand
- (C) It would make Push $O(N)$
- (D) Arrays of pointers cannot be dynamically allocated

95. What is the better solution proposed for a stack of unbounded size?

- (A) A larger static array
- (B) Chaining the elements one after another
- (C) Two parallel arrays
- (D) A sorted array

96. Once the elements are chained, what is the only thing needed to reach the whole stack?

- (A) The maxQue value
- (B) The top pointer
- (C) An index counter
- (D) Both a front and a rear pointer

97. What are the two members of NodeType?

- (A) info and prev
- (B) info and next
- (C) data and index
- (D) key and value

98. What does the 'info' member of NodeType hold?

- (A) A pointer to the next node
- (B) The user's data
- (C) The node's address
- (D) The size of the node

99. Given 'NodeType<char> node, *nodePtr;', which pair of statements is correct?

- (A) node->info = 'D'; nodePtr.info = 'L';
- (B) node.info = 'D'; nodePtr->info = 'L';
- (C) node.info = 'D'; nodePtr.info = 'L';
- (D) node->info = 'D'; nodePtr->info = 'L';

100. Before 'nodePtr->info = 'L';' can be executed safely, what must happen?

- (A) Nothing - nodePtr is ready to use
- (B) nodePtr must be given a node, e.g. nodePtr = new NodeType<char>;
- (C) nodePtr must be set to NULL
- (D) info must be declared static

101. In the linked-list stacktype.h, where is struct NodeType declared?

- (A) Globally, before the class
- (B) Inside class StackType
- (C) Inside stacktype.cpp only
- (D) Inside the public section as a typedef

102. What is the only private data member of the linked-list StackType?

- (A) int top
- (B) ItemType items[MAX_ITEMS]
- (C) NodeType<ItemType>* topPtr
- (D) NodeType<ItemType>* front

103. Which member function appears in the array-based StackType but NOT in the linked-list StackType header?

- (A) Push
- (B) Top
- (C) MakeEmpty
- (D) IsFull

104. Which member function appears in the linked-list StackType but NOT in the array-based one?

- (A) IsEmpty
- (B) The destructor ~StackType()
- (C) Push
- (D) Top

105. What does the linked-list StackType constructor do?

- (A) topPtr = new NodeType;
- (B) topPtr = NULL;
- (C) top = -1;
- (D) Allocates MAX_ITEMS nodes

106. What is the body of IsEmpty() in the linked-list stack?

- (A) return (topPtr == NULL);
- (B) return (top == -1);
- (C) return (topPtr->next == NULL);
- (D) return (topPtr != NULL);

107. What does Top() return in the linked-list stack?

- (A) topPtr
- (B) topPtr->next
- (C) topPtr->info
- (D) *topPtr

108. How is IsFull() implemented in the linked-list stack?

- (A) By comparing a counter with MAX_ITEMS
- (B) By attempting 'location = new NodeType;' inside a try block and catching std::bad_alloc
- (C) By checking whether topPtr == NULL
- (D) By returning false always

109. Which exception type does IsFull() catch?

- (A) FullStack
- (B) EmptyStack
- (C) std::bad_alloc
- (D) std::out_of_range

110. Inside IsFull(), why is 'delete location;' executed immediately after the successful allocation?

- (A) To set location to NULL
- (B) Because the node was only a test allocation and keeping it would leak memory
- (C) Because delete is required before return
- (D) To trigger the catch block

111. What does IsFull() return when the trial allocation succeeds?

- (A) true
- (B) false
- (C) NULL
- (D) It throws FullStack

112. In Push, what is the correct order of the last two pointer assignments?

- (A) topPtr = location; then location->next = topPtr;
- (B) location->next = topPtr; then topPtr = location;
- (C) location->next = NULL; then topPtr = location;
- (D) topPtr->next = location; then topPtr = location;

113. What happens if the two assignments in Push are written in the wrong order (topPtr = location; location->next = topPtr;)?

- (A) Nothing - the result is identical
- (B) The new node points to itself and the rest of the stack becomes unreachable (memory leak)
- (C) An EmptyStack exception is thrown
- (D) The stack is reversed

114. In Push, what value is stored in location->info?

- (A) topPtr->info
- (B) newItem
- (C) NULL
- (D) The address of the previous node

115. What is the time complexity of Push in the linked-list stack?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N)$ only when the stack is full

116. Which local pointer does Pop() declare, and why?

- (A) location, to allocate a new node
- (B) tempPtr, to hold the old top node so it can be deleted after topPtr is moved
- (C) front, to mark the bottom of the stack
- (D) None - Pop needs no local pointer

117. What is the correct sequence of statements inside Pop() (after the empty check)?

- (A) delete topPtr; topPtr = topPtr->next;
- (B) tempPtr = topPtr; topPtr = topPtr->next; delete tempPtr;
- (C) topPtr = topPtr->next; tempPtr = topPtr; delete tempPtr;
- (D) tempPtr = topPtr->next; delete topPtr; topPtr = tempPtr->next;

118. What would happen if Pop() executed 'delete topPtr;' before advancing topPtr?

- (A) Nothing unusual
- (B) topPtr->next would be read from deleted memory, so the rest of the stack would be lost
- (C) An EmptyStack exception would be thrown
- (D) The stack would become full

119. What does Pop() do differently from the array version regarding memory?

- (A) It does nothing to memory
- (B) It actually returns the node's memory to the free store with delete
- (C) It reallocates the whole stack
- (D) It calls MakeEmpty

120. What is the time complexity of Pop in the linked-list stack?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N^2)$

121. How does the linked-list StackType destructor work?

- (A) It calls delete [] on the node array
- (B) It loops while (topPtr != NULL), saving, advancing and deleting each node
- (C) It sets topPtr = NULL only
- (D) It calls Pop exactly MAX_ITEMS times

122. What is the time complexity of the linked-list StackType destructor?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N \log N)$

123. Which operation of the linked-list stack is NOT $O(1)$?

- (A) Push
- (B) Pop
- (C) Top
- (D) The destructor

124. Comparing the two stack implementations, which statement is correct?

- (A) Both need MAX_ITEMS known in advance
- (B) The array version wastes no memory but is size-limited; the linked version is size-flexible but stores an extra pointer per element
- (C) The linked version has $O(N)$ Push
- (D) The array version needs a destructor

SECTION E - Queue: Linked-List-Based Implementation

125. What is the stated advantage of the linked-list queue over the array-based queue?

- (A) It is faster than $O(1)$
- (B) The queue grows and shrinks on demand, with no size fixed beforehand
- (C) It does not need exception classes
- (D) It removes the need for a front pointer

126. What are the two private data members of the linked-list QueType?

- (A) front and rear indices of type int
- (B) `NodeType<ItemType>* front` and `NodeType<ItemType>* rear`
- (C) `topPtr` and `maxQue`
- (D) `items` and `maxQue`

127. Which member of the array-based QueType is completely absent from the linked-list QueType?

- (A) `IsEmpty`
- (B) `maxQue`
- (C) `Enqueue`
- (D) The destructor

128. What does the linked-list QueType constructor do?

- (A) `front = rear = 0;`
- (B) `front = NULL; rear = NULL;`
- (C) `front = new NodeType; rear = front;`
- (D) `front = maxQue - 1; rear = maxQue - 1;`

129. What is the body of `IsEmpty()` in the linked-list queue?

- (A) `return (rear == front);`
- (B) `return (front == NULL);`
- (C) `return (rear == NULL && front != NULL);`
- (D) `return (front->next == NULL);`

130. How is IsFull() implemented in the linked-list queue?

- (A) By comparing a counter to DEFAULT_SIZE
- (B) Exactly like the linked stack: a trial 'new NodeType' inside try/catch on std::bad_alloc
- (C) By checking $(\text{rear}+1)\% \text{maxQue} == \text{front}$
- (D) It always returns true

131. In Enqueue, what value is assigned to newNode->next?

- (A) front
- (B) rear
- (C) NULL
- (D) newNode

132. What is the purpose of the test 'if (rear == NULL)' inside Enqueue?

- (A) To check whether memory is exhausted
- (B) To detect an empty queue, in which case the new node is also the front node
- (C) To decide whether to throw FullQueue
- (D) To wrap the rear index around

133. In Enqueue, what happens in the else branch (when rear is not NULL)?

- (A) `front = newNode;`
- (B) `rear->next = newNode;`
- (C) `newNode->next = rear;`
- (D) `delete rear;`

134. Which statement in Enqueue is executed in BOTH the empty and non-empty cases?

- (A) `front = newNode;`
- (B) `rear->next = newNode;`
- (C) `rear = newNode;`
- (D) `delete newNode;`

135. What is the time complexity of Enqueue in the linked-list queue?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N)$ when the queue is empty

136. Why is a rear pointer kept at all, rather than walking to the end of the list on each Enqueue?

- (A) To distinguish empty from full
- (B) To keep Enqueue $O(1)$ instead of $O(N)$
- (C) Because Dequeue needs it
- (D) To allow the queue to wrap around

137. What is the correct statement order inside Dequeue (after the empty check)?

- (A) `front = front->next; item = front->info; delete front;`
- (B) `tempPtr = front; item = front->info; front = front->next; if (front == NULL) rear = NULL; delete tempPtr;`
- (C) `item = rear->info; rear = rear->next; delete rear;`
- (D) `delete front; front = front->next; item = front->info;`

138. Why does Dequeue contain 'if (front == NULL) rear = NULL;'?

- (A) To free the last node
- (B) Because when the last node is removed, rear would otherwise be left dangling at a deleted node
- (C) To throw EmptyQueue
- (D) To reset the item parameter

139. What would go wrong if the 'if (front == NULL) rear = NULL;' line were omitted?

- (A) Dequeue would become $O(N)$
- (B) After emptying the queue, rear would point to freed memory and the next Enqueue would corrupt the queue
- (C) IsEmpty would always return false
- (D) front would never advance

140. What is the time complexity of Dequeue in the linked-list queue?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N^2)$

141. How does MakeEmpty() work in the linked-list queue?

- (A) front = rear = maxQue - 1;
- (B) It loops while (front != NULL) deleting each node, then sets rear = NULL
- (C) It calls delete [] on the node array
- (D) It sets front = NULL only

142. What is the time complexity of MakeEmpty() in the linked-list queue?

- (A) $O(1)$
- (B) $O(N)$
- (C) $O(\log N)$
- (D) $O(N \log N)$

143. What does the linked-list QueType destructor do?

- (A) Nothing
- (B) It calls MakeEmpty()
- (C) It executes delete [] items;
- (D) It sets front and rear to NULL

144. Why is MakeEmpty $O(1)$ for the array queue but $O(N)$ for the linked queue?

- (A) Because the linked queue is slower at arithmetic
- (B) The array queue only resets two indices; the linked queue must individually free every dynamically allocated node
- (C) Because the array queue has no destructor
- (D) Because the linked queue has two pointers

145. A linked queue is empty. After Enqueue('A'), what are the values of front and rear?

- (A) Both NULL
- (B) Both point to the node containing 'A'
- (C) front points to 'A', rear is NULL
- (D) front is NULL, rear points to 'A'

146. After Enqueue('A'), Enqueue('B'), Enqueue('C'), which node does rear point to?

- (A) A
- (B) B
- (C) C
- (D) NULL

147. Continuing from A, B, C, after one Dequeue the front pointer points to the node containing:

- (A) A
- (B) B
- (C) C
- (D) NULL

148. Which end of the linked queue does Enqueue operate on, and which does Dequeue operate on?

- (A) Enqueue at front, Dequeue at rear
- (B) Enqueue at rear, Dequeue at front
- (C) Both at the front
- (D) Both at the rear

149. In the linked-list queue, is the reserved-slot trick from the array implementation still required?

- (A) Yes, one node is always reserved
- (B) No - empty is simply front == NULL, so there is no ambiguity
- (C) Yes, but only when the queue is full
- (D) Only when ItemType is char

SECTION F - Recursion: Concepts, Recursion vs Iteration

150. Which definition of recursion matches the slides?

- (A) A technique that repeats a block of code using a loop
- (B) A technique that solves a problem by solving smaller versions of the same problem
- (C) A technique that divides memory between the stack and the heap
- (D) A technique that stores results of subproblems in a table

151. When a recursive technique is turned into a program, you end up with:

- (A) Functions that call themselves
- (B) Functions with no return value
- (C) Functions with global variables
- (D) Nested loops

152. What is the stated advantage of recursive algorithms?

- (A) They always run faster than loops
- (B) They can simplify the solution, giving shorter, more easily understood source code
- (C) They use less memory
- (D) They avoid the need for base cases

153. What is the stated disadvantage of recursive algorithms?

- (A) They cannot express every algorithm
- (B) They are often less efficient than iterative solutions in both time and space
- (C) They cannot be used with templates
- (D) They always cause infinite loops

154. Iteration achieves repetition using which C++ structures?

- (A) if, if...else, switch
- (B) for, while, do...while
- (C) try and catch
- (D) new and delete

155. Recursion achieves repetition using which C++ structures?

- (A) for, while, do...while
- (B) Selection structures: if, if...else or switch
- (C) goto statements
- (D) Exception handlers

156. Iteration terminates when:

- (A) The base case is satisfied
- (B) The loop-continuation condition fails
- (C) The stack overflows
- (D) The counter becomes negative

157. Recursion terminates when:

- (A) The loop-continuation condition fails
- (B) The base case is satisfied
- (C) The counter reaches zero
- (D) The activation record is popped

158. Iteration controls repetition by:

- (A) Dividing the problem into simpler ones
- (B) Using a counter
- (C) Pushing activation records
- (D) Throwing exceptions

159. Recursion controls repetition by:

- (A) Using a counter
- (B) Dividing the problem into simpler one(s)
- (C) Using a while loop
- (D) Restarting the function

160. In the general form of a recursive function, what happens in the base case?

- (A) The function calls itself once more
- (B) The problem is solved directly, without recursion
- (C) The function returns void
- (D) The problem is decomposed further

161. In the general form of a recursive function, the subproblems produced in the general case must be:

- (A) Of the same size as the original
- (B) Similar and strictly smaller than the original
- (C) Larger than the original
- (D) Completely unrelated to the original

162. What happens if a recursive function has no reachable base case?

- (A) It returns 0
- (B) It recurses without end, consuming the run-time stack until the program fails
- (C) The compiler converts it to a loop
- (D) It returns the last computed value

163. The recursive definition of $n!$ given in the slides is:

- (A) $n! = 1$ if $n = 0$; $n! = (n-1)! * n$ if $n > 0$
- (B) $n! = 0$ if $n = 0$; $n! = n! * (n-1)$ if $n > 0$
- (C) $n! = n$ if $n = 1$; $n! = n * (n+1)!$ if $n > 1$
- (D) $n! = 1 * 2 * 3 * \dots * n$

164. The closed form solution of $n!$ given in the slides is:

- (A) 1 if $n = 0$; $(n-1)! * n$ if $n > 0$
- (B) 1 if $n = 0$; $1*2*3*\dots*(n-1)*n$ if $n > 0$
- (C) $n * (n-1)$ for all n
- (D) n^2 for all n

165. In the recursive Factorial function, which line is the base case?

- (A) `return n * Factorial(n-1);`
- (B) `if (n == 0) return 1;`
- (C) `int Factorial(int n)`
- (D) `else`

166. In the recursive Factorial function, which line decomposes, solves and combines?

- (A) `if (n == 0)`
- (B) `return 1;`
- (C) `return n * Factorial(n-1);`
- (D) `int Factorial(int n)`

167. Tracing Factorial(5) as shown in the slides, the final value returned is:

- (A) 24
- (B) 120
- (C) 720
- (D) 25

168. In the Factorial trace, which value is returned first, before any multiplication happens?

- (A) 120
- (B) 24
- (C) 1, returned by the $0!$ base case
- (D) 5

169. The three steps for writing a recursive function, in order, are:

- (A) Verify, decompose, code
- (B) Determine the size factor; determine the base case(s); determine the general case(s); then verify
- (C) Write the loop, remove the loop, add a call
- (D) Determine the general case, then the base case, then the counter

170. What is meant by the 'size factor' of a recursive problem?

- (A) The number of lines in the function
- (B) The measure of problem size that shrinks with each recursive call
- (C) The amount of memory the function uses
- (D) The number of base cases

171. The Base-Case Question of the Three-Question Verification Method asks:

- (A) Is there a non-recursive way out of the function, and does the routine work correctly for that base case?
- (B) Does each call use less memory?
- (C) Is the base case the first statement?
- (D) Are there at least two base cases?

172. The Smaller-Caller Question asks:

- (A) Is the function shorter than the iterative version?
- (B) Does each recursive call involve a smaller case of the original problem, leading inescapably to the base case?
- (C) Does the caller pass parameters by reference?
- (D) Is the recursion depth shallow?

173. The General-Case Question asks:

- (A) Whether the general case runs before the base case
- (B) Assuming the recursive call(s) work correctly, does the whole function work correctly?
- (C) Whether the general case can be replaced by a loop
- (D) Whether the function has a return type

174. Which of the three verification questions is essentially an assumption of correctness, similar to an inductive hypothesis?

- (A) The Base-Case Question
- (B) The Smaller-Caller Question
- (C) The General-Case Question
- (D) None of them

175. According to the slides, a recursive solution should be preferred when:

- (A) The recursion depth is very deep
- (B) The recursive version is shorter and simpler, the depth of calls is relatively shallow, and it does about the same amount of work as the non-recursive version
- (C) The problem involves arrays
- (D) Memory is limited

176. The Comb(6,4) call tree is used in the slides to illustrate:

- (A) That recursion always beats iteration
- (B) That plain recursion can recompute the same subproblems many times, making it very inefficient
- (C) How to write a base case
- (D) How activation records are popped

177. Which technique is presented as the way to avoid recomputing the same subproblems?

- (A) Tail recursion
- (B) Dynamic programming
- (C) Exception handling
- (D) Circular arrays

178. What is the core idea of dynamic programming, as stated in the slides?

- (A) Replace all recursion with loops
- (B) Store the results of subproblems so that they need not be recomputed when needed later
- (C) Increase the size of the run-time stack
- (D) Use references instead of pointers

179. According to the slides, dynamic programming is studied in detail in which course?

- (A) CSE 225
- (B) CSE 373
- (C) CSE 115
- (D) CSE 331

SECTION G - Recursion: The Run-Time Stack and Function Calls

180. How does the computer implement recursion?

- (A) By copying the function's code for each call
- (B) By using a stack called the 'run-time' stack to keep track of the function calls
- (C) By using a circular queue of return addresses
- (D) By converting recursion into iteration automatically

181. What is created and stored in the run-time stack each time a function is called recursively?

- (A) A new copy of the source code
- (B) An activation record
- (C) A node in a linked list
- (D) An exception object

182. What happens to the activation record when the recursion returns?

- (A) It is pushed again
- (B) It is popped out of the stack
- (C) It is copied to the heap
- (D) It is left in place until the program ends

183. Which data structure discipline does the run-time stack follow?

- (A) FIFO
- (B) LIFO
- (C) Priority order
- (D) Random order

184. During the call $z = a(y)$ inside function b , which of the following must be saved?

- (A) Only the return value of a
- (B) Everything about b that will be needed later - its variables x, y, z and the place to resume execution
- (C) The entire source file
- (D) Nothing, since a has its own variables

185. In the call $z = a(y)$ where a is declared as $\text{int } a(\text{int } w)$, what binding occurs?

- (A) w is bound to z
- (B) The argument passed from b is bound to the parameter w of a
- (C) y is bound to x
- (D) a is bound to b

186. After function a finishes executing, what happens?

- (A) Function b restarts from its first line
- (B) The activation record is popped, b 's old values are restored, and a 's return value replaces $a(x)$ in the assignment
- (C) The run-time stack is cleared entirely
- (D) b is pushed onto the stack again

187. What is the difference between a recursive call and a non-recursive call, from the run-time system's point of view?

- (A) Recursive calls skip the activation record
- (B) There is no difference
- (C) Recursive calls use the heap instead of the stack
- (D) Recursive calls do not bind parameters

188. For the function: `int f(int x){ int y; if(x==0) return 1; else { y = 2*f(x-1); return y+1; } }`, what is `f(0)`?

- (A) 0
- (B) 1
- (C) 2
- (D) 3

189. For the same function `f`, what is `f(1)`?

- (A) 2
- (B) 3
- (C) 4
- (D) 5

190. For the same function `f`, what is `f(2)`?

- (A) 6
- (B) 7
- (C) 8
- (D) 9

191. For the same function `f`, what is `f(3)`?

- (A) 7
- (B) 14
- (C) 15
- (D) 16

192. In the trace of `f(3)`, how many copies of `f` are pushed onto the run-time stack in total?

- (A) Two
- (B) Three
- (C) Four
- (D) Eight

193. In the trace of `f(3)`, in which order are the values returned?

- (A) 15, 7, 3, 1
- (B) 1, 3, 7, 15
- (C) 1, 2, 6, 14
- (D) 3, 7, 15, 1

194. What is the stated conclusion about the cost of recursion?

- (A) Recursion is always faster than iteration
- (B) Recursion could be very slow because of the extra memory and time overhead due to function calls
- (C) Recursion uses no extra memory
- (D) Recursion cannot be traced

195. Where does the extra **SPACE overhead of recursion mainly come from?**

- (A) The heap allocations made by `new`
- (B) The activation records stacked up on the run-time stack, one per pending call
- (C) The size of the source code
- (D) The exception classes

SECTION H - Recursion Applied: Binary Search and Sorted Linked Lists

196. In the iterative RetrievalItem, what is the loop-continuation condition?

- (A) (first < last)
- (B) (first <= last) && !found
- (C) (midPoint != item)
- (D) (found == true)

197. In both versions of binary search, how is midPoint computed?

- (A) (first + last) / 2
- (B) (last - first) / 2
- (C) length / 2
- (D) (first + last + 1) / 2

198. In the iterative version, what happens when item < info[midPoint]?

- (A) first = midPoint + 1;
- (B) last = midPoint - 1;
- (C) found = true;
- (D) midPoint--;

199. What is the size factor for recursive binary search?

- (A) The value of item
- (B) The number of elements in (info[first] ... info[last])
- (C) The value of midPoint
- (D) The length of the whole array, always

200. How many base cases does the recursive binary search have?

- (A) One
- (B) Two
- (C) Three
- (D) None

201. What is base case 1 of recursive binary search?

- (A) If item == info[midPoint], return true
- (B) If first > last, return false
- (C) If first == last, return true
- (D) If length == 0, throw an exception

202. What is base case 2 of recursive binary search?

- (A) If first > last, return false
- (B) If item == info[midPoint], return true
- (C) If midPoint == 0, return false
- (D) If item > info[last], return false

203. What is general case 1 (item < info[midPoint])?

- (A) return BinarySearch(info, item, midPoint+1, last);
- (B) return BinarySearch(info, item, first, midPoint-1);
- (C) return BinarySearch(info, item, first, last);
- (D) return false;

204. What is general case 2 (item > info[midPoint])?

- (A) return BinarySearch(info, item, first, midPoint-1);
- (B) return BinarySearch(info, item, midPoint+1, last);
- (C) return BinarySearch(info, item, midPoint, last);
- (D) return true;

205. Why do the general cases use midPoint-1 and midPoint+1 rather than midPoint?

- (A) To avoid array bounds errors
- (B) Because info[midPoint] has already been compared, so excluding it guarantees a strictly smaller subproblem
- (C) Because midPoint is always even
- (D) To keep first <= last

206. What is the body of the recursive version of Retrieveltem?

- (A) It contains the whole search loop
- (B) found = BinarySearch(info, item, 0, length-1);
- (C) return BinarySearch(info, item, 0, length);
- (D) It calls Insert(listData, item);

207. Why is a separate wrapper function such as Retrieveltem used?

- (A) To make the code longer
- (B) Because the recursive function needs extra parameters (first and last) that the public interface should not expose
- (C) Because member functions cannot recurse
- (D) To catch exceptions

208. What is the size factor for the recursive InsertItem on a sorted list?

- (A) The value of newItem
- (B) The number of elements in the current list
- (C) The number of pointers in the node
- (D) The length of the array

209. How many base cases does the recursive Insert have, and what are they?

- (A) One: location == NULL
- (B) Two: the list is empty (location == NULL), or item < location->info
- (C) Two: item == location->info, or location == NULL
- (D) Three

210. What is the general case of the recursive Insert?

- (A) Insert(location, item);
- (B) Insert(location->next, item);
- (C) Insert(listData, item);
- (D) return;

211. In the recursive Insert, the parameter is declared as NodeType<ItemType>* &location. What does the '&' achieve?

- (A) It passes the node by value
- (B) It makes location an alias for the actual pointer variable in the caller, so assigning to location updates the real link in the list
- (C) It converts the pointer to a reference to ItemType
- (D) It prevents the node from being deleted

212. In the recursive Insert, what does 'location' become an alias of during the recursive calls?

- (A) listData only
- (B) listData at the top level, and N1->next, N2->next, ... in successive calls
- (C) The item being inserted
- (D) The head of a new list

213. Which pointer, needed in the non-recursive insertion, is NOT needed in the recursive version?

- (A) listData
- (B) predLoc (the trailing/predecessor pointer)
- (C) location
- (D) tempPtr

214. Why is predLoc unnecessary in the recursive insertion?

- (A) Because the list is sorted
- (B) Because the reference parameter already IS the predecessor's next field
- (C) Because insertion always happens at the head
- (D) Because the list has no predecessors

215. Inside the base case of recursive Insert, what is the correct order of statements?

- (A) location = new NodeType; tempPtr = location; location->next = tempPtr;
- (B) tempPtr = location; location = new NodeType<ItemType>; location->info = item; location->next = tempPtr;
- (C) location->info = item; tempPtr = location; location = new NodeType<ItemType>;
- (D) delete location; location = new NodeType<ItemType>;

216. What is the size factor for the recursive DeleteItem on a sorted list?

- (A) The value of item
- (B) The number of elements in the list
- (C) The number of base cases
- (D) The length of the array

217. What is the base case of the recursive Delete?

- (A) location == NULL
- (B) item == location->info
- (C) item < location->info
- (D) location->next == NULL

218. What is the general case of the recursive Delete?

- (A) Delete(listData, item);
- (B) Delete(location->next, item);
- (C) Delete(location, item->next);
- (D) delete location;

219. Inside the base case of the recursive Delete, what is the correct sequence?

- (A) delete location; location = location->next;
- (B) tempPtr = location; location = location->next; delete tempPtr;
- (C) location = location->next; tempPtr = location; delete tempPtr;
- (D) tempPtr = location->next; delete location;

220. The recursive Delete as written assumes:

- (A) The item may or may not be present
- (B) The item is definitely present in the list, since there is no location == NULL base case
- (C) The list is unsorted
- (D) The list has exactly one node

221. Which structural feature makes both recursive Insert and recursive Delete work without a trailing pointer?

- (A) The list being sorted
- (B) Passing the node pointer by reference (NodeType<ItemType>* &location)
- (C) The use of templates
- (D) The use of exception classes

ANSWER KEY WITH EXPLANATIONS

1. **(B)** Last In First Out (LIFO) — Lecture 08, slide 2: a stack is a list where data items can be added and deleted, maintaining LIFO order.
2. **(C)** The top of the stack — The 'Structure' entry of the specification states elements are added to and removed from the top.
3. **(B)** Definitions provided by the user — The specification splits into 'Definitions (provided by user)' - MAX_ITEMS and ItemType - and 'Operations (provided by the ADT)'.
4. **(B)** The maximum number of items that might be on the stack — MAX_ITEMS is the maximum number of items that might be on the stack.
5. **(A)** The data type of the items on the stack — ItemType is the data type of the items on the stack.
6. **(C)** MakeEmpty — MakeEmpty only lists 'Postcondition: Stack is empty'. All others require the stack to have been initialized.
7. **(B)** The stack has been initialized — Every operation except MakeEmpty lists 'Precondition: Stack has been initialized'.
8. **(C)** Exception FullStack is thrown — Push postcondition: if (stack is full), exception FullStack is thrown, else newItem is at the top of the stack.
9. **(B)** Pop and Top — The comment in stacktype.h states EmptyStack is the exception class used by Pop and Top when the stack is empty.
10. **(C)** ItemType — The specification declares 'ItemType Top()' - it returns a copy of the top item.
11. **(B)** Pop removes the top item, Top returns a copy of it without removing — Pop removes the top item (returns nothing); Top returns a copy of the top item and leaves the stack unchanged.
12. **(B)** 5 — stacktype.h contains: const int MAX_ITEMS = 5;
13. **(B)** As empty classes used as exception classes — Both are declared as 'class FullStack {};' and 'class EmptyStack {};' - empty classes used purely as exception types.
14. **(B)** int top and ItemType items[MAX_ITEMS] — private: int top; ItemType items[MAX_ITEMS];
15. **(B)** To act as an include guard preventing multiple inclusion — STACKTYPE_H_INCLUDED is a header include guard.
16. **(B)** The class is generic and works with any item type supplied at compile time — It makes StackType a class template parameterised by ItemType.
17. **(B)** The items array is a static member array, so no dynamic memory is allocated — items[MAX_ITEMS] is an ordinary member array; nothing is allocated with new, so nothing must be freed.
18. **(B)** -1 — StackType<ItemType>::StackType() { top = -1; }
19. **(B)** Because -1 signals that no element occupies index 0 yet; the first Push makes top 0 — Push increments top before storing, so top = -1 makes the first stored index 0.
20. **(B)** return (top == -1); — return (top == -1);
21. **(B)** return (top == MAX_ITEMS - 1); — return (top == MAX_ITEMS-1); - the last valid index is MAX_ITEMS-1.
22. **(B)** top = -1; — MakeEmpty simply resets top = -1; the old data is not erased, just made unreachable.
23. **(B)** top++; then items[top] = newItem; — top++ first moves top to the next free slot, then items[top] = newItem stores there.

24. **(B)** Checks if(IsFull()) and throws FullStack() — if(IsFull()) throw FullStack();
25. **(C)** Only decrements top, leaving the old value physically in the array but unreachable — Pop is just 'top--' after the empty check - the data is logically removed, not physically erased.
26. **(B)** items[top] — return items[top]; after checking IsEmpty().
27. **(B)** 2 — top goes -1 -> 0 -> 1 -> 2.
28. **(B)** 4 — IsFull() returns top == MAX_ITEMS-1 == 4.
29. **(C)** An EmptyStack exception — Pop checks IsEmpty() first and throws EmptyStack().
30. **(A)** O(1) — Slide 7 marks every array-based stack operation as O(1).
31. **(D)** None - all of them are O(1) — Constructor, IsEmpty, IsFull, MakeEmpty, Push, Pop and Top are all marked O(1).
32. **(B)** Because it only assigns top = -1 and does not touch the stored elements — A single assignment takes constant time regardless of how many items were on the stack.
33. **(B)** Initialise an empty stack — Step 1: Initialise an empty stack.
34. **(B)** It is pushed into the stack — Step 2(a): If the item is an opening parenthesis, push it into the stack.
35. **(B)** A pop is performed on the stack — Step 2(b): Else, if the item is a closing parenthesis, pop from the stack.
36. **(B)** A balanced string of parentheses — Empty stack indicates a balanced string of parentheses.
37. **(B)** An unbalanced string of parentheses — Non-empty stack indicates an unbalanced string of parentheses.
38. **(B)** An unbalanced string of parentheses — An unsuccessful pop indicates an unbalanced string of parentheses - a closing parenthesis with no matching opener.
39. **(C)** (()) (()) () — Only (()) (()) () has every opener matched by a later closer with none left over.
40. **(B)** The most recently opened parenthesis is the first one that must be closed (LIFO) — Nesting is inherently last-opened-first-closed, which is exactly LIFO behaviour.
41. **(B)** Two unmatched opening parentheses — The trailing (() leaves two openers unmatched, so the stack is non-empty and the string is unbalanced.
42. **(B)** No parentheses are necessary — Slide 29 asks 'Why? No parentheses necessary!'
43. **(A)** 2 3 4 * - 5 + — From the table on slide 29.
44. **(B)** -5 — $3*4 = 12$; $2-12 = -10$; $-10+5 = -5$.
45. **(B)** 2 3 - 4 5 + * — From the table on slide 29.
46. **(B)** -9 — $(-1) * 9 = -9$.
47. **(C)** -15 — $3*4 = 12$; $12+5 = 17$; $2-17 = -15$. Its infix form is $2 - (3*4 + 5)$.
48. **(B)** A binary tree — Slide 30: 'Use binary tree to convert the following expression ... into Postfix form'.
49. **(A)** 4 8 + 6 5 - * 3 2 - 2 2 + * / — Exactly as given on slide 30.
50. **(B)** 3 — $(12)*(1) = 12$; $(1)*(4) = 4$; $12/4 = 3$.
51. **(A)** It is pushed into the stack — Step 2(a): If the item is an operand, push it into the stack.
52. **(C)** The top two items are popped, the operator is applied, and the answer is pushed back — Step 2(b) of the postfix evaluation algorithm.
53. **(B)** A valid expression, and that item is the answer — Only item in stack: indicates valid expression.
54. **(B)** An invalid expression — More than one item in stack or unsuccessful pop: indicates invalid expression.
55. **(B)** 12 — 4 and 8 are popped, $4+8 = 12$ is pushed back, so the stack holds only 12.
56. **(A)** 12 and 1 — 6 and 5 are popped, $6-5 = 1$ is pushed, leaving 12 1.
57. **(B)** Because - and / are not commutative, so swapping the operands changes the result — The second-popped value is the left operand; reversing it turns 6-5 into 5-6.
58. **(B)** First In First Out (FIFO) — Lecture 08, slide 50: a queue maintains First In First Out order.

- 59. (B)** Added to the rear and removed from the front — Structure: elements are added to the rear and removed from the front of the queue.
- 60. (C)** Top — A queue has no Top operation; its operations are MakeEmpty, IsEmpty, IsFull, Enqueue and Dequeue.
- 61. (C)** void Dequeue(ItemType& item) — void Dequeue(ItemType& item) - the removed element is returned through the reference parameter.
- 62. (B)** Because the function returns void and must pass the removed element back through the parameter — Dequeue's postcondition says the front element is removed and item is a copy of the removed element.
- 63. (B)** The EmptyQueue exception is thrown and item is undefined — If (queue is empty), EmptyQueue exception is thrown and item is undefined.
- 64. (B)** The FullQueue exception is thrown — If (queue is full), FullQueue exception is thrown, else newItem is at rear of queue.
- 65. (C)** O(N) — Slide 53: 'Complexity of deletion: O(N)' because every remaining element must be shifted one position left.
- 66. (B)** Dead space accumulates at the front of the array, which can never be reused — Slide 54 labels the unusable slots left behind by front++ as 'Dead space'.
- 67. (B)** rear = (rear + 1) % maxQue; — For Enqueue: rear = (rear + 1) % maxQue; where maxQue is the array size.
- 68. (B)** front = (front + 1) % maxQue; — For Dequeue: front = (front + 1) % maxQue;
- 69. (B)** Reusing the dead space at the beginning of the array — Wrapping around lets the queue reuse slots freed at the front of the array.
- 70. (B)** Because both situations can produce the same relative values of front and rear — Slide 56 shows a dequeue-to-empty and an enqueue-to-full that produce identical front/rear values.
- 71. (B)** Let front indicate the index of the array slot PRECEDING the front element and leave that slot reserved — Slide 57: front indicates the slot preceding the front element, and that slot is reserved - items are never stored there.
- 72. (B)** One array slot is always wasted, so the array must be one larger than the capacity — This is why the constructor sets maxQue = max + 1.
- 73. (B)** (rear + 1) % maxQue == front — Slide 58: Full queue when (rear + 1) % maxQue == front.
- 74. (A)** front == rear — Slide 58: Empty queue when front == rear.
- 75. (C)** 500 — #define DEFAULT_SIZE 500
- 76. (B)** ItemType is an alias for char in this version of the code — typedef creates the alias ItemType for the built-in type char.
- 77. (B)** int front, int rear, ItemType* items, int maxQue — private: int front; int rear; ItemType* items; int maxQue;
- 78. (B)** Two - a default one and one taking int max — QueType(); and QueType(int max);
- 79. (B)** To account for the one reserved slot that is never filled — The reserved slot means the physical array must be one larger than the logical capacity.
- 80. (C)** maxQue - 1 — front = maxQue - 1; rear = maxQue - 1;
- 81. (C)** Executes delete [] items; — ~QueType() { delete [] items; } - the items array was allocated with new[].
- 82. (B)** Because QueType allocates its array dynamically with new ItemType[maxQue] — StackType uses a fixed member array; QueType uses a dynamically allocated array that must be released.
- 83. (B)** front = maxQue - 1; rear = maxQue - 1; — MakeEmpty resets both indices to maxQue - 1, restoring the initial empty state.
- 84. (B)** return (rear == front); — return (rear == front);
- 85. (B)** rear = (rear + 1) % maxQue; then items[rear] = newItem; — rear is advanced first, then the item is written at the new rear position.
- 86. (B)** front = (front + 1) % maxQue; then item = items[front]; — Because front points to the slot BEFORE the front element, it must be advanced first, then read.

- 87. (C)** Both $O(1)$ — Slide 63 marks both as $O(1)$ - the modulo arithmetic replaces any shifting.
- 88. (A)** $O(1)$ — Slide 62 marks 'delete [] items;' as $O(1)$ - it is a single deallocation of one block.
- 89. (B)** $\text{maxQue} = 5$, capacity 4 — $\text{maxQue} = \text{max} + 1 = 5$, and one slot stays reserved, so 4 items can be stored.
- 90. (B)** Full — $(\text{rear} + 1) \% \text{maxQue} = 2 == \text{front}$, which is the full condition.
- 91. (B)** Empty — $\text{front} == \text{rear}$ is the empty condition.
- 92. (B)** You must anticipate the size of the stack before you use it — Lecture 09, slide 2: in either case you need to anticipate the size of the stack before you use it.
- 93. (B)** In the free store (heap) — The slide labels the allocated cell as 'Somewhere in free store'.
- 94. (B)** You would still need to know the size of that array beforehand — Slide 6: even if the pointer array is dynamically allocated, its size still has to be known in advance.
- 95. (B)** Chaining the elements one after another — Slide 7: 'A better solution is chaining the elements one after another.'
- 96. (B)** The top pointer — Slide 7: 'All we need is the top pointer.'
- 97. (B)** info and next — `struct NodeType { ItemType info; NodeType* next; };`
- 98. (B)** The user's data — info holds the user's data; next holds a pointer to the next node.
- 99. (B)** `node.info = 'D'; nodePtr->info = 'L';` — Dot operator for an object, arrow operator for a pointer to an object.
- 100. (B)** `nodePtr` must be given a node, e.g. `nodePtr = new NodeType<char>;` — The slide shows '`nodePtr = new NodeType<char>;`' before the assignment.
- 101. (B)** Inside class `StackType` — `NodeType` is declared as a nested struct at the top of class `StackType`.
- 102. (C)** `NodeType<ItemType>* topPtr` — private: `NodeType<ItemType>* topPtr;`
- 103. (C)** `MakeEmpty` — The linked version declares `StackType()`, `~StackType()`, `Push`, `Pop`, `Top`, `IsEmpty` and `IsFull` - there is no `MakeEmpty`.
- 104. (B)** The destructor `~StackType()` — The linked version needs a destructor to release all dynamically allocated nodes.
- 105. (B)** `topPtr = NULL;` — `StackType<ItemType>::StackType() { topPtr = NULL; }`
- 106. (A)** `return (topPtr == NULL);` — An empty linked stack is one whose top pointer is `NULL`.
- 107. (C)** `topPtr->info` — After the `IsEmpty` check, it returns `topPtr->info`.
- 108. (B)** By attempting '`location = new NodeType;`' inside a try block and catching `std::bad_alloc` — There is no fixed capacity, so fullness means memory exhaustion, detected by a trial allocation.
- 109. (C)** `std::bad_alloc` — `catch(std::bad_alloc& exception) { return true; }`
- 110. (B)** Because the node was only a test allocation and keeping it would leak memory — The node serves only to prove memory is available; it is released at once.
- 111. (B)** `false` — Success means memory is still available, so the stack is not full: `return false`.
- 112. (B)** `location->next = topPtr;` then `topPtr = location;` — The new node must first be linked to the old top, then `topPtr` is moved to the new node.
- 113. (B)** The new node points to itself and the rest of the stack becomes unreachable (memory leak) — Once `topPtr` has been overwritten, the address of the old top is lost and `location->next` points back at `location`.
- 114. (B)** `newItem` — `location->info = newItem;`
- 115. (A)** $O(1)$ — Slide 26 marks `Push` as $O(1)$ - a fixed number of pointer assignments, no traversal.
- 116. (B)** `tempPtr`, to hold the old top node so it can be deleted after `topPtr` is moved — `tempPtr = topPtr;`
`topPtr = topPtr->next;` `delete tempPtr;`
- 117. (B)** `tempPtr = topPtr;` `topPtr = topPtr->next;` `delete tempPtr;` — Save the old top, advance `topPtr`, then delete the saved node.
- 118. (B)** `topPtr->next` would be read from deleted memory, so the rest of the stack would be lost — The link to the remainder of the stack lives inside the node being freed.

- 119. (B)** It actually returns the node's memory to the free store with delete — The array version only decrements top; the linked version frees the node.
- 120. (A)** $O(1)$ — Slide 38 marks Pop as $O(1)$.
- 121. (B)** It loops while (`topPtr != NULL`), saving, advancing and deleting each node — A while loop walks the chain deleting each node in turn.
- 122. (B)** $O(N)$ — Slide 40 marks it $O(N)$ - every one of the N nodes must be visited and deleted.
- 123. (D)** The destructor — The destructor is $O(N)$; everything else is $O(1)$.
- 124. (B)** The array version wastes no memory but is size-limited; the linked version is size-flexible but stores an extra pointer per element — This is the essential trade-off between the two implementations.
- 125. (B)** The queue grows and shrinks on demand, with no size fixed beforehand — Lecture 09, slide 41.
- 126. (B)** `NodeType<ItemType>* front` and `NodeType<ItemType>* rear` — private: `NodeType<ItemType>* front`; `NodeType<ItemType>* rear`;
- 127. (B)** `maxQue` — With no array there is no capacity value, so `maxQue` and the modulo arithmetic disappear.
- 128. (B)** `front = NULL`; `rear = NULL`; — Both pointers start as `NULL`, meaning an empty queue.
- 129. (B)** `return (front == NULL)`; — `return (front == NULL)`;
- 130. (B)** Exactly like the linked stack: a trial 'new `NodeType`' inside try/catch on `std::bad_alloc` — The same trial-allocation technique is used for both linked structures.
- 131. (C)** `NULL` — `newNode->next = NULL`; because the new node becomes the last node of the chain.
- 132. (B)** To detect an empty queue, in which case the new node is also the front node — If the queue was empty there is no `rear->next` to attach to, so `front` must be set to the new node.
- 133. (B)** `rear->next = newNode`; — The old last node is linked to the new node.
- 134. (C)** `rear = newNode`; — `rear = newNode`; sits outside the if/else - the new node always becomes the rear.
- 135. (A)** $O(1)$ — Slide 65 marks it $O(1)$ - the rear pointer removes any need to traverse the list.
- 136. (B)** To keep Enqueue $O(1)$ instead of $O(N)$ — Without rear, each insertion would require traversing the whole chain.
- 137. (B)** `tempPtr = front`; `item = front->info`; `front = front->next`; if (`front == NULL`) `rear = NULL`; delete `tempPtr`; — Save the node, copy out its data, advance front, fix rear if the queue emptied, then delete.
- 138. (B)** Because when the last node is removed, rear would otherwise be left dangling at a deleted node — front becoming `NULL` means the queue is now empty, so rear must also be reset to `NULL`.
- 139. (B)** After emptying the queue, rear would point to freed memory and the next Enqueue would corrupt the queue — Enqueue tests `rear == NULL` to detect an empty queue; a stale rear breaks that test.
- 140. (A)** $O(1)$ — Slide 83 marks it $O(1)$.
- 141. (B)** It loops while (`front != NULL`) deleting each node, then sets `rear = NULL` — A while loop frees every node, and rear is set to `NULL` at the end.
- 142. (B)** $O(N)$ — Slide 85 marks it $O(N)$ - every node must be individually deleted.
- 143. (B)** It calls `MakeEmpty()` — `~QueueType() { MakeEmpty(); }` - which makes the destructor $O(N)$ as well.
- 144. (B)** The array queue only resets two indices; the linked queue must individually free every dynamically allocated node — Resetting indices is one operation; releasing N heap nodes is N operations.
- 145. (B)** Both point to the node containing 'A' — Because `rear == NULL`, `front = newNode` is executed, and `rear = newNode` follows.
- 146. (C)** C — rear always points to the most recently enqueued node.
- 147. (B)** B — Dequeue removes A and advances front to B.
- 148. (B)** Enqueue at rear, Dequeue at front — This preserves FIFO order.
- 149. (B)** No - empty is simply `front == NULL`, so there is no ambiguity — The empty/full ambiguity was an artefact of circular index arithmetic and disappears with pointers.
- 150. (B)** A technique that solves a problem by solving smaller versions of the same problem — Lecture 10, slide 2.

- 151. (A)** Functions that call themselves — That is the definition of a recursive function.
- 152. (B)** They can simplify the solution, giving shorter, more easily understood source code — Slide 3.
- 153. (B)** They are often less efficient than iterative solutions in both time and space — Slide 3: '...they often less efficient, both in terms of time and space, than non-recursive (e.g., iterative) solutions.'
- 154. (B)** for, while, do...while — Slide 4: iteration uses repetition structures.
- 155. (B)** Selection structures: if, if...else or switch — Slide 5: recursion uses selection structures and repeated function calls.
- 156. (B)** The loop-continuation condition fails — Slide 4.
- 157. (B)** The base case is satisfied — Slide 5.
- 158. (B)** Using a counter — Slide 4.
- 159. (B)** Dividing the problem into simpler one(s) — Slide 5.
- 160. (B)** The problem is solved directly, without recursion — The base case is where the problem is minimal / not decomposable and is solved directly.
- 161. (B)** Similar and strictly smaller than the original — Step (1): decompose the problem into one or more similar, STRICTLY SMALLER subproblems.
- 162. (B)** It recurses without end, consuming the run-time stack until the program fails — Every call pushes another activation record and none is ever popped.
- 163. (A)** $n! = 1$ if $n = 0$; $n! = (n-1)! * n$ if $n > 0$ — The last option is the closed form, not the recursive definition.
- 164. (B)** 1 if $n = 0$; $1*2*3*...*(n-1)*n$ if $n > 0$ — The closed form expresses the answer directly, without referring to $n!$ again.
- 165. (B)** if $(n == 0)$ return 1; — if $(n==0)$ return 1; is marked '// base case' in the slide.
- 166. (C)** return $n * \text{Factorial}(n-1)$; — $\text{Factorial}(n-1)$ decomposes and solves; multiplying by n combines.
- 167. (B)** 120 — $5! = 5 * 24 = 120$.
- 168. (C)** 1, returned by the 0! base case — The innermost call $1 * 0!$ returns 1, and the multiplications are then completed on the way back up.
- 169. (B)** Determine the size factor; determine the base case(s); determine the general case(s); then verify — Slide 17.
- 170. (B)** The measure of problem size that shrinks with each recursive call — It is the quantity you must show is getting smaller, e.g. the number of elements in `info[first]..info[last]`.
- 171. (A)** Is there a non-recursive way out of the function, and does the routine work correctly for that base case? — Slide 18, question 1.
- 172. (B)** Does each recursive call involve a smaller case of the original problem, leading inescapably to the base case? — Slide 18, question 2.
- 173. (B)** Assuming the recursive call(s) work correctly, does the whole function work correctly? — Slide 18, question 3.
- 174. (C)** The General-Case Question — It begins 'Assuming that the recursive call(s) work correctly...'
- 175. (B)** The recursive version is shorter and simpler, the depth of calls is relatively shallow, and it does about the same amount of work as the non-recursive version — Slide 30 lists exactly these three criteria.
- 176. (B)** That plain recursion can recompute the same subproblems many times, making it very inefficient — Slide 31: 'Recursion could be very inefficient!'
- 177. (B)** Dynamic programming — Slide 31: 'Dynamic programming can avoid this issue!'
- 178. (B)** Store the results of subproblems so that they need not be recomputed when needed later — Slide 31 describes DP as mainly an optimization over plain recursion.
- 179. (B)** CSE 373 — The slide ends with 'Learn it in CSE 373.'
- 180. (B)** By using a stack called the 'run-time' stack to keep track of the function calls — Slide 10.
- 181. (B)** An activation record — Slide 10.
- 182. (B)** It is popped out of the stack — Slide 10: when recursion returns, the corresponding activation record is popped out of the stack.

- 183. (B)** LIFO — It is a stack, so the most recent call is the first to be completed and removed.
- 184. (B)** Everything about `b` that will be needed later - its variables `x`, `y`, `z` and the place to resume execution — Slide 12, step 2.
- 185. (B)** The argument passed from `b` is bound to the parameter `w` of `a` — The actual argument is bound to the formal parameter before control transfers.
- 186. (B)** The activation record is popped, `b`'s old values are restored, and `a`'s return value replaces `a(x)` in the assignment — Slide 13.
- 187. (B)** There is no difference — Slide 14: 'There is no difference between recursive and non-recursive calls!'
- 188. (B)** 1 — `x == 0` hits the base case and returns 1.
- 189. (B)** 3 — `y = 2*f(0) = 2`; return `y+1 = 3`.
- 190. (B)** 7 — `y = 2*f(1) = 6`; return `y+1 = 7`.
- 191. (C)** 15 — `y = 2*f(2) = 14`; return `y+1 = 15`, matching the trace on slide 15.
- 192. (C)** Four — `f(3)`, `f(2)`, `f(1)` and `f(0)` - four activation records, popped in reverse order.
- 193. (B)** 1, 3, 7, 15 — The innermost call finishes first: `f(0)=1`, then `f(1)=3`, `f(2)=7`, `f(3)=15`.
- 194. (B)** Recursion could be very slow because of the extra memory and time overhead due to function calls — Slide 16.
- 195. (B)** The activation records stacked up on the run-time stack, one per pending call — Each pending call keeps a full activation record alive until it returns.
- 196. (B)** `(first <= last) && !found` — `while( (first <= last) && !found)`
- 197. (A)** `(first + last) / 2` — `midPoint = (first + last) / 2`;
- 198. (B)** `last = midPoint - 1`; — Searching moves into the lower half.
- 199. (B)** The number of elements in `(info[first] ... info[last])` — Slide 20.
- 200. (B)** Two — Base case 1: `first > last`, return false. Base case 2: `item == info[midPoint]`, return true.
- 201. (B)** If `first > last`, return false — It represents an exhausted search range - the item is not present.
- 202. (B)** If `item == info[midPoint]`, return true — The item has been found at the midpoint.
- 203. (B)** `return BinarySearch(info, item, first, midPoint-1)`; — Search the first half, excluding the midpoint.
- 204. (B)** `return BinarySearch(info, item, midPoint+1, last)`; — Search the second half, excluding the midpoint.
- 205. (B)** Because `info[midPoint]` has already been compared, so excluding it guarantees a strictly smaller subproblem — Including `midPoint` again could make the range never shrink, so the base case might never be reached.
- 206. (B)** `found = BinarySearch(info, item, 0, length-1)`; — `RetrieveItem` is a thin wrapper that supplies the initial first and last values.
- 207. (B)** Because the recursive function needs extra parameters (`first` and `last`) that the public interface should not expose — The user calls `RetrieveItem(item, found)`; the wrapper supplies 0 and `length-1`.
- 208. (B)** The number of elements in the current list — Slide 24.
- 209. (B)** Two: the list is empty (`location == NULL`), or `item < location->info` — `if ((location == NULL) || (item < location->info)) { ... } // base cases`
- 210. (B)** `Insert(location->next, item)`; — It moves one node further down the list.
- 211. (B)** It makes `location` an alias for the actual pointer variable in the caller, so assigning to `location` updates the real link in the list — This is why the calls can be written as `Insert(location->next, item)` and still modify the caller's next field.
- 212. (B)** `listData` at the top level, and `N1->next`, `N2->next`, ... in successive calls — The diagram shows `location` as an alias of `listData`, then of `N1->next`, `N2->next`, `N3->next`.
- 213. (B)** `predLoc` (the trailing/predecessor pointer) — Slide 26: 'Note: no `predLoc` pointer is needed for insertion!'
- 214. (B)** Because the reference parameter already IS the predecessor's next field — The recursion carries the link that must be modified, so no separate trailing pointer is needed.

215. (B) `tempPtr = location; location = new NodeType<ItemType>; location->info = item; location->next = tempPtr;` — The old sublist is saved in `tempPtr` first, then the new node is created and linked in front of it.

216. (B) The number of elements in the list — Slide 27.

217. (B) `item == location->info` — If `item == location->info`, delete the node pointed to by `location`.

218. (B) `Delete(location->next, item);` — Slide 27.

219. (B) `tempPtr = location; location = location->next; delete tempPtr;` — Save the node, bypass it by advancing the aliased link, then free the saved node.

220. (B) The item is definitely present in the list, since there is no `location == NULL` base case — There is no guard against running off the end, so an absent item would dereference `NULL`.

221. (B) Passing the node pointer by reference (`NodeType<ItemType>* &location`) — Reference-to-pointer is the key mechanism in both functions.